

Vore War: Community Content Guide

1st edition – January 3, 2026

0th edition – July 16, 2023

Preface

About This Book

This guidebook first started when I, CaneSugarCat, wrote a step-by-step guide on how to create a game option to remind myself of the process should I take a break and forget. It began on a simple plaintext document. Then, when I finally decided it was time to share these varied guides, realized not everyone was on the same level as me in terms of computer knowledge. I couldn't just write down steps, I needed file paths, and Unity references. I figured I would need some screenshots and code blocks to make it an official guide. Not to mention spell check and formatting. So, here we are, an entire book guide full of ways to add your own awesome content to the game.

Prerequisites

While this book will instruct and guide you through the codebase, there are some things I recommend you know or learn before attempting to add your own content to the game. This is to reduce your own frustration when reading. Coding specific topics will be colored in **red**, so if you only intend to create sprites, you can safely ignore them.

Despite what it may say below, the **red** requirements themselves are not 100% required. I began implementing the Day/Night cycle long before I knew C#, and while I was still new to Python. I was able to get the fog of war toggle to alternate every turn, and eventually even made a setting for it in the menu. If you're determined enough and know how to do a little copy/paste coding and a few changes here and there, you'll go a lot farther than you may think.

In this guide, I assume at least moderate familiarity with the following topics:

- **Basic Computer knowledge:** You'll need to know how to use your computer. This includes using your File Explorer, understanding what a file path looks like, how to copy/paste/duplicate/extract/open files, and how to install programs.
- **Determination and Attention to detail:** These two things are a must, no matter what you intend to do. One line of code can throw you off and waste hours of time, or your art may not be at the level you want to share. Be prepared to fail and pick yourself up again, because in the end, it will (hopefully) be worth it. Things take time, and nobody is in a rush.
- **Git:** Git is an important tool for sharing your creations, as pull requests are required to merge your changes into the main repo. You can learn more about how to set up Git on your computer here: <https://docs.github.com/en/get-started/quickstart/set-up-git>
- **Unity:** While it is not a requirement to know everything about Unity, it will be important to familiarize yourself with the windows within. These windows will be referred to in the guides, so knowing the names of each will help you quickly navigate.
- **C#:** C# is what the game is coded in. If you don't know it, it's not completely detrimental, but you may spend more time fixing your code than writing new things, depending on what you're trying to do.
- **OOP:** At least basic knowledge of object-oriented programming will be 100% required and assumed. You will need to know how to identify and define various data types, how to

duplicate and modify code, and how to use conditionals. How to inherit a class, how to use an inherited class, and shorthand coding is very useful, but not as required.

- **Mathematical maturity:** You must know how to add, subtract, multiply and divide, and have some exposure to trigonometry, or at the very least know how to operate a calculator to do so for you.

Additional References

These guides are obviously only one way to add content to the game. Authors and readers bring their own perspectives to any intellectual material. If you don't understand one set of instructions don't be shy to ask on the discord. A few people on the development team, those with an orange name, can help you out. There's also a few people who don't have an orange name who can help as well, poke around #coding-discussion and see who's talking smart.

Acknowledgements

This guidebook is mainly drawn from my experience in coding for VW.

I am grateful to:

darkevilme for the initial game.

Aurilika, Selica, Greenslime, and Micadi, for really taking off with the concept.

Cattitude for taking up the mantel after Auri's EOS.

Flame_Valxsarion and Scarabyte for being cool and helping with my initial contributions.

And you, the reader, who hopes to add their great ideas to the game.

A Lighthearted Disclaimer

Despite my best efforts, these guides may or may not contain several mistakes, errors, typos, failures, bugs, blunders, exaggerations, ramblings, repetitions, face plants, distortions, tomfoolery, oversimplifications, redundancy, outright lies, repetitions, trash, nonsense and even misinformation.

Ping me on the VW Discord if there are any issues, grammatical errors, or questions about the guide. And feel free to do the same for any additions you want to see, or even general feedback. It's all good stuff.

You can also DM me these things, but you'll have to either include a picture of a cat (any kind) or a delicious looking consumable with over 50 grams of added sugars, just so I know you mean business.

Can't wait to see what you add and thanks for reading!

-CaneSugarCat

Table of Contents

Vore War: Community Content Guide	i
Introduction	2
How to start	2
Making Sprites.....	5
Requesting your additions	9
Coding	10
Introduction.....	10
Coding a race.....	10
Coding a trait	26
Making options	28

Introduction

How to start

Before starting, if you intend to share your changes or additions, which I hope you would, it's important that you set up Git. If you only intend to make sprites, you don't worry about setting up Unity and can skip to the section [Downloading the Project](#).

If you do intend to write code, I recommend an IDE that supports Git integration, such as Microsoft Visual Studio (Code or 20XX), so you don't have to fool around with the command line. If you really don't want to download an IDE for whatever reason, I'd recommend you download Notepad++ so you at least have context coloring and autofill. How you choose to write your code up to you and specifics on the topic are not brought up.

You'll need to set up the Unity project, this needs to be done if you intend to play an experimental build or add things to the game, though not necessarily required if you intend to just make sprites. Why don't we get started?

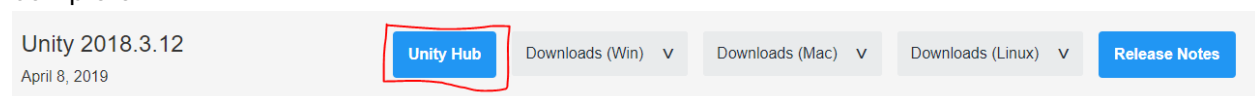
Downloading Unity

To run Unity, you'll need to first download one of two programs:

1. **Unity Hub:** A program that handles your Unity versions and has a GUI which allows you to see what projects you have downloaded, useful if you have multiple projects.
2. **Standard Unity:** Is just the unity program, I personally use Unity Hub, but this option is lighter if you're stingy about disk space or don't want to bother with other programs.

Go to this link <https://unity.com/releases/editor/archive>

Click the 'Unity 2018 X' tab and scroll down to 2018.3.12, either click the blue Unity Hub Version if you downloaded it, or the 'Unity Installer' under your desired OS. Then, wait for your download to complete.



While Unity is downloading, let's move onto the next step.

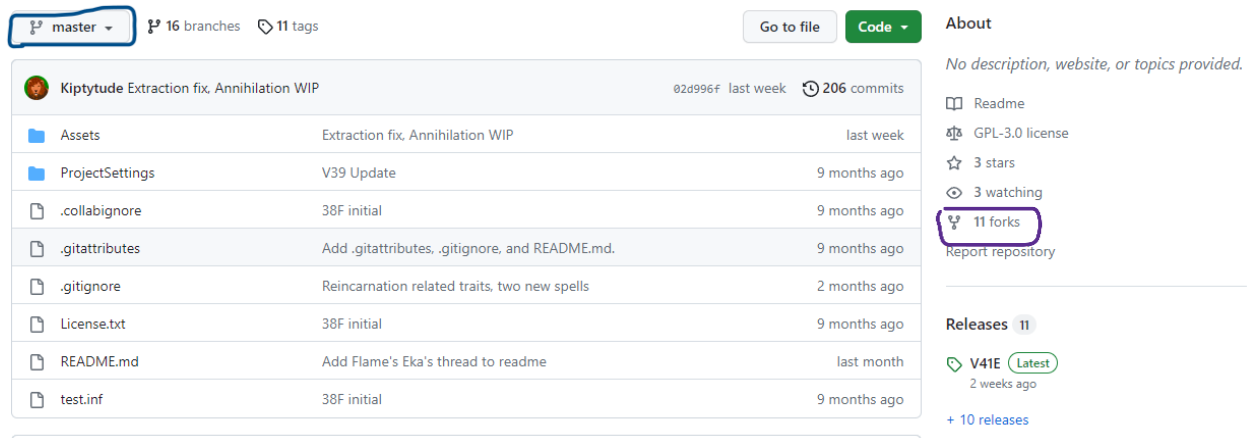
Downloading the Project

We need to download the VW project from the main repository. This repository holds everything you need to add what you want. Essentially, it's an unbuilt project.

Navigate to the main GitHub page here: <https://github.com/Flame-Valxsarion/VoreWar> There will be a few points of interest:

1. **Forks:** Forks are where you can find work in progress repositories from the community that are not yet released. Usually, but not always, the GitHub username matches their Discord name.

2. **Branch Selector:** This selects the branch of the current repo, branches tend to be different versions of the game, the current build can be seen under releases, and is the 'master' branch.
3. **Code Button:** The code button is where you can download the current repo, as a zip or using the git URL. If you're using an IDE with git integration, use its integrated functionality to clone.



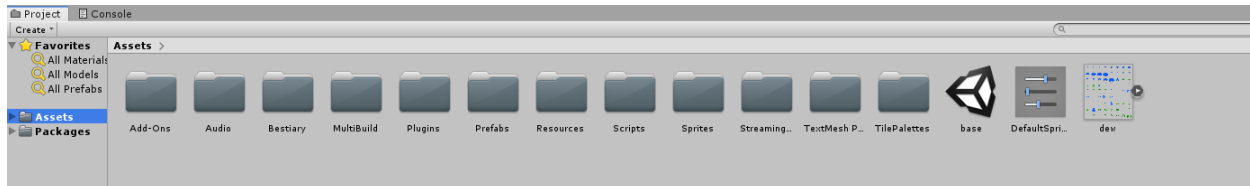
Download the repo using your desired method once you've navigated to the desired fork and branch. Place the zip or clone the repository somewhere easy to access and remember, you'll be going back two to three times during the setup. And that's it, you've successfully downloaded the game's source code!

If you only trying to make sprites for the game, feel free to head down to the [Making Sprites](#) section and get reading there.

Loading Up the Project

Depending on if you downloaded Unity Hub or not this process varies:

- **Unity Hub:**
 - Click the '**Add**' Button, then navigate to where you put your download.
 - Click the VoreWar-*** folder, *** is the name of the branch you selected above.
 - Select the VoreWar project.
 - After Unity finishes loading look at the bottom most '**project**' window, click the Assets folder on the left, if it's not already selected.
 - Double click **Base**.



- **Unity Program:**
 - Click **File -> Open Scene**
 - Navigate to where you put your download.

- Open the VoreWar-*** folder, *** is the name of the branch you selected above.
- Open the **Assets** folder.
- Click **base.unity**, then open.

Your project should now be loaded up. You can now run the game in the Unity editor.

Additional notes

Now that you have a loaded project into Unity you can work with it or build the source into a program like the release. If you have your own GitHub repository, you can even make a release and have people try the features you made early to help you iron out bugs.

Here's how you build the game in five easy steps:

1. Click **File -> Build Settings...** or **Ctrl+Shift+B**
2. Confirm base is checked.
3. Confirm your platform is correct.
4. Click '**Build**' or '**Build and Run**'
5. Select a folder.

You now have the game as an .exe in the folder you selected, ready to be zipped and distributed.

If you chose Visual Studio as your IDE, you'll want a solution to work with. If you have Microsoft Visual Studio installed, Unity will make a solution for you. Here's how:

1. In Unity, open the scripts folder in the bottom 'projects' window.
2. Double click any of the scripts
3. Wait for Unity to open that file in VS.
4. Close VS.
5. Navigate to where you put your repository download.
6. A **.sln** file will be there, which you can double click to open your solution.

Making Sprites

Introduction

One thing that is often asked in Discord is how to make your own unit or race in the game. There are many ways to make sprites. Whether you're an experienced artist or are completely to it, there are a few things that need to be addressed before you jump into making sprites.

Software

First, you'll need your very own software. This software can realistically be anything that manipulates an image, I'd recommend something you're familiar with, but don't be afraid to try something new. I'd consider it a requirement for you to find software that supports layers, as the game renders sprites in layers. Many people use the following programs to make their sprites:

- Aseprite
- GIMP
- PaintTool SAI
- Paint.net
- Adobe Photoshop

I've personally used both Aseprite and GIMP, and they work well. Feel free to ask on Discord what others use if these options don't appeal to you, I'm sure there's at least one or two others not listed here. Once you have your preferred software all set up, you can get started. Feel free to keep reading for more information though.

References

If you're interested in the already made sprites used by the game, they are in the GitHub repository. You'll need to navigate to the already created sprites folder to find them.

If you downloaded the repo, these sprite images will be located at **VoreWar/Assets/Sprites/**.

If for whatever reason you didn't download the repo, this is the link for those who don't feel like digging around.

<https://github.com/Kiptytude/VoreWar/tree/master/Assets/Sprites>

Final Set Up

Now that you have your desired software, and some references, you had better bring your creative side out, because it's nearly time to get started. Here's a bit more information to help you when making sprites.

Create a new file in your editor and select a size for your sprite. Many units are **160x160**, this is the standard for most empire races, but not all. One notable empire race is the equine, which has 50% more resolution due to their larger size at **240x240**. Then there are dragons, which have two times the "standard" resolution at **320x320**. There is no real limit to how large your sprite can be, and it doesn't need to be square either, as Abakhanskya is a hefty **860x538**!

Once you have selected a suitable size, you can go about the next step in a few different ways. There is no correct or wrong way, and the ways listed are not the only ways you can go about making your sprites.

1. **Sprite sheet:** Where a set of sprites are lined next to each other in a sheet. Sprites are made in a similar way to reference files and are organized by sheets. With this method, make sure the size of your sheet is a multiple of the sprite size you've chosen.
2. **Individual Images:** Where each sprite is its own individual image, in the size you chose. I personally don't use this method, but it can be useful for some. However, I do know you'll need to pack the sheet after you're done.

If you chose to make a **Sprite Sheet**, there's a bit more set up to ensure your sprites stay aligned. It's important to create a grid based on your sprite size. Depending on your program, you can use rulers, use a grid, or change the transparent background to the size of your sprite. Here are the ways I currently know how:

- **Aseprite - changing the background:** ctrl + k -> Background -> Size -> Custom -> Change both values to your sprite size.
- **Aseprite- making a grid:** View -> Grid -> Grid Settings -> Change Width and height.
- **Gimp - making a grid:** (View -> Show Grid) then (Image -> Configure Grid)

No matter what you use as your grid, make sure you keep your sprites lined up with how they should look in game. While not a complete detriment if they aren't, paying a bit more attention while creating the sprites can help save you time tweaking positions when you're done.

Now everything is ready for you to get started making sprites. Remember, it takes time, there's no need to rush. Pace yourself and take breaks when you need to, the last thing you want is to burn out and scrap your project.

Additional Notes

While you have everything you need now to make sprites, here is a collection of notes to help you make sprites. Read these notes to get some insight into the practices of myself and a few others. I mainly follow the current art style of the game, but it's not a requirement. In fact, I encourage you to express your style. There are plenty of examples where someone branched off already included in the game.

Below are some of the ways you can help keep your sprites consistent with the game's current style.

Colors

Most colors have 4 shades. One as the base, a highlight (50% lighter), a shadow (50% darker), and an outline (70-95% darker). This is not strict, but this has been the general convention for most sprites in the game. Feel free to use more shades or less shades, it's completely up to you. If your program doesn't support pallets, make your own on your sprite sheet to easily reference back to and use the dropper tool on. Just remember to delete it once you complete the sprite sheet.



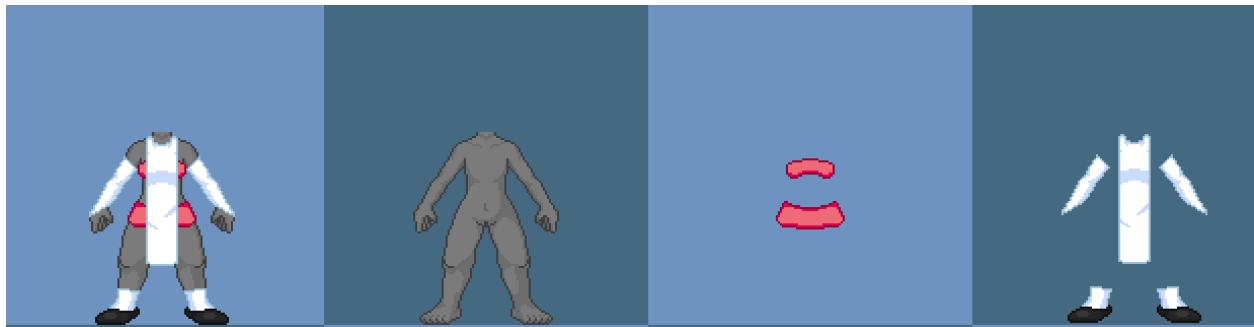
An example of the Aseprite pallet for Mice



An example of an on-sheet pallet for inferno_the_dragon's Orcas

Sprite layers

Use layers to sprite over bodies or clothing if the clothing has multiple layers. This will keep everything aligned. If creating on a sprite sheet, you can use your marquee tool and select the entire grid square to move your sprite to another. Additionally, placing different parts of clothing on other layers makes them easy to change later.



There's no shame in finding references of things such as clothing, weapons, and concepts online. There are many things you can base your sprite off, even if you only really want one thing from the image. Obviously don't take someone's OC without asking because I doubt, they would be very happy if they found out.

If you don't want to sprite from scratch, use already made sprite sheets as a reference. There are already examples of pants, skirts and other clothing items already spirited. You can save some time by locating them and using them as a base or taking parts from them, such as a belt.

Changing parts

When making sprites, keep in mind what parts of your race are going to move and change, to reduce the sprites you need to do, split up your clothes and body parts in a way that keeps each moving part independent. Here's an example below.



On the left is a clothing sprite for the mice, while on the right is some of their possible body sprites. Notice how many pieces of clothes are separate sprites due to how the legs and arms change on the right body. When making female clothing both the breast area and belly area (of not naturally exposed) will need to be separated as there a large amount of both belly and breast sprites that the combinations would get out of hand very quickly.

Checklist

And that's pretty much all the tips and notes I have from my experience. If you have anything you would like others to know, you know where to find me, feel free to speak up and ping. I'll leave you with a checklist for what each type of common sprite needs:

Unit:

- Any kind of body sprites
- A palette for their colors (Placed into **VoreWar\Assets\Resources\ColorPalettes**)

Main Race:

- **32 x 32** Village Sprites (appended to **VoreWar\Assets\Sprites\strategicvillages.png**)
 - There are three sprites, the first is a color changing part for team indication, the second is the base village, and the third is the walled village.
 - I recommend you start by making the base village, then figuring out what you want to indicate the team color and how you want your wall to look.
- **64 x 64** Banner Icon Sprite (appended to **VoreWar\Assets\Sprites\Banners.png**)
- A optional palette for their colors (Placed into **VoreWar\Assets\Resources\ColorPalettes**)

Monster:

- Monster Icon (appended to **VoreWar\Assets\Sprites\ strategicsprites.png**)

Closing thoughts

Making sprites takes time, there's no need to rush and burn yourself out. Take breaks for days, weeks or even months if you need to, nobody is going to like you any less for it (I'd hope).

Feel free to post your work in progress, concepts, or completed sprites in #sprite-sharing on Discord. Everyone is looking forward to new content, and it's an easy way to bring some excitement from time to time or get feedback.

Lastly, remember that these are your sprites. Nobody has the right to judge their quality because there's a good chance your critic is not making any themselves. Take your time, practice, iterate, and improve; You only stay in one place if you don't make the strides to another. Best of luck to ya!

Requesting your additions

If you've completed your sprites, have no idea how to code and don't wish to try at all, there is still hope. You can ask someone to code them for you. The way to do this is up to you really; As I have just one point of view. Below is how I would request my sprites be added and how I would like to be approached if someone were to ask me to add the sprites they made. I can't provide a clear cut or uniform way of doing this that everyone will accept, as I've never requested one myself.

To start, I'd provide some information about the race, including all of these:

- Name of your race.
- Body size of your race, keep in mind base races average size 10 and dragons are size 100.
- Stomach size of your race, keep in mind base races average size 16 and dragons are size 80.
- Allowed vore types for your race, essentially any type you made sprites for.
- Your race's favored stat
- The default stat spread of your race, might be overruled if it's a main race
- Default traits
- Optionally, a description of your race

Remember to be nice when asking, the developers do it out of their own good will. It takes a fair amount of time to code and test so be patient. Many active devs are likely working on their own projects.

Coding

Introduction

So, you're looking to add your own content or code in your race yourself? Good choice. As stated before, the game uses the C# language.

Coding a race

Alright, it's time to code in a main race. This process takes your sprite sheet and combines all your individual parts into a complete, layered sprite. Initially, this process can look rather daunting, but this guide will make the process tedious at worst, as there is not much logic that goes into sprite coding.

Prep Work

Navigate to VoreWar\Assets\Scripts\Races

To start, I recommend copying an existing race file and renaming it to your own race. This is an easy way to ensure you have all the required fields. Copying a related file can greatly reduce the work you need to do, especially if your sprites use the same positions as existing ones. If your file is missing context or there are errors pertaining to types or namespaces, open the project in Unity and allow it to make a meta file.

Starting on your newly copied race file, replace the names of the main class with the name of your race. Then, do the same with the function below.

```
class Mice : DefaultRaceData
{
    . . .
    public Mice()
    {
```

Should become

```
class YOUR_RACE : DefaultRaceData
{
    . . .
    public YOUR_RACE()
    {
```

Next, we'll be opening many different files and copy/paste coding. We do this because many aspects of the main race files are defined in the files below.

Navigate to VoreWar\Assets\Scripts\Races\Races.cs

Add your race by following the present implementations.

Navigate to VoreWar\Assets\Scripts\Enums.cs

Add your race to the end of the Enum list for main races.

IMPORTANT: Your main race should be added beneath Vagrants, ideally right after the latest.

Navigate to VoreWar\Assets\Scripts\Races\Graphics\SpriteDictionary.cs

Follow the present implementation and add your sprites above the 'Unique Mercenaries' header, these use the names of your .png files, so you can copy/paste.

Navigate to VoreWar\Assets\Scripts\Races\RaceParameters.cs

Follow the present implementation and add your race.

Be mindful to keep your race in the same area as the like races, as main races, mercs, monsters, and special mercs are defined in their own 'blocks'. In other words, add your race after the last race of what you're adding. For example, Mice are inserted after Aabax, despite it being in the middle of the document.

Add your race to the switch statement below.

Scroll down to [RaceParameters\(\)](#).

Within [RaceParameters\(\)](#), scroll to the same area as your race and add your entry directly after the latest one, copy/pasting if need be. **Adjust 'BodySize' and 'StomachSize'**, to your liking, where empire races average 10 and 16 respectively. Add your desired traits to your race. Try to stay around those values, but don't feel too constrained if your race doesn't conform.

Optionally, add a description for your race, you can read others to get an idea for what to write.

Sprite Definition

We're now done with all the above files, so you're free to close them. **Return to your race file.** There's a bit of setup that's required before we get into the real work.

At the top of your file, just below the class definition, **replace all 'Sprite[]' code with the ones you added in SpriteDictionary.cs**. This should only require you to change the last bit on the right of the equal sign. It might be useful to replace some of the variable names as well, though this is not required.

Example:

Ambiguous Names

```
1. readonly Sprite[] Sprites = State.GameManager.SpriteDictionary.Mice1;
2. readonly Sprite[] Sprites2 = State.GameManager.SpriteDictionary.Mice2;
3. readonly Sprite[] Sprites3 = State.GameManager.SpriteDictionary.MiceFBottoms;
4. readonly Sprite[] Sprites4 = State.GameManager.SpriteDictionary.MiceFTops;
5. readonly Sprite[] Sprites5 = State.GameManager.SpriteDictionary.MiceMBottoms;
6. readonly Sprite[] Sprites6 = State.GameManager.SpriteDictionary.MiceMTops;
7. readonly Sprite[] Sprites7 = State.GameManager.SpriteDictionary.MiceVore1;
8. readonly Sprite[] Sprites8 = State.GameManager.SpriteDictionary.MiceVore2;
9. readonly Sprite[] Sprites9 = State.GameManager.SpriteDictionary.MiceVore3;
```

Specific Names

```
1. readonly Sprite[] SpritesBelliesBalls= State.GameManager.SpriteDictionary.DemidragonBelliesBalls;
2. readonly Sprite[] SpritesBodies1 = State.GameManager.SpriteDictionary.DemidragonBodies;
3. readonly Sprite[] SpritesBodies2 = State.GameManager.SpriteDictionary.DemidragonBodies2;
4. readonly Sprite[] SpritesBreasts = State.GameManager.SpriteDictionary.DemidragonBreasts;
5.
```

Part Definition

With that, everything has now been set up, and we can begin. Looking in the code, there's several variables that may be hard to understand at first glance. I'll list them out here, as well as their function.

'oversize' - Used for breasts larger than the maximum size, usually these have a sprite associated with it.

'BodySizes' - The count for weight gain sprites.

'BodyAccentTypesX' - Additions to your sprites such as antlers or patterns, these are usually and should be marked with a comment so you can keep track of what they are. You can have up to five different BodyAccentTypes. Types such as Head, Tail, Ear, and Fur, have their own slots, so you can use them instead of BodyAccentTypes. There are 15 body accent types.

If you created a furry version of your race, ensure **'FurCapable = true;'** is present, if it's not, add it.

Next up, you'll see a large block of seemingly repeated code, starting with **'Body = ...'**. These are the definitions of the main sprites, set equal to a `SpriteExtraInfo()` function. We'll be looking at what each of the function's arguments mean.

Example of part definitions:

```
1. Body = new SpriteExtraInfo(4, BodySprite, null, (s) => FurryColor(s));
2. Head = new SpriteExtraInfo(6, HeadSprite, null, (s) => FurryColor(s));
3. BodyAccent = new SpriteExtraInfo(4, BodyAccentSprite, null, (s) => FurryColor(s)); // Right Arm
4. BodyAccent2 = new SpriteExtraInfo(3, BodyAccentSprite2, null, (s) => FurryColor(s)); // Tail
```

The **first** argument given to the function is the layer, essentially, what layer the sprite will be rendered on. Higher layers will be placed on top of lower ones. The **second** argument is a Sprite returning function defined in your file. If you need, you can write additional functions, just remember they need to return the 'Sprite' data type. Remember these functions, as we will come back to them later. The **third** argument is for a specific color, usually, this is set to 'null' and can be ignored. The **fourth** argument is for the palette, which we will also come back to.

Clothing Definition

Below the large block of parts are clothes that you do not want automatically generated on units. These clothes are defined outside of the main list. Specific clothes are separated if they need to be specifically referenced by outside functions, such as the clothes for leaders or the rags clothing type.

```
1. LeaderClothes = new MiceLeader();
2. Rags = new MiceRags();
```

Note: These are classed exactly as any other clothes.

Add the following to the bottom of your race file for each clothing item.

```
class YOUR_CLOTHING_ITEM : MainClothing
{
    public YOUR_CLOTHING_ITEM()
    {
    }

    public override void Configure(CompleteSprite sprite, Actor_Unit actor)
    {
    }
}
```

Replace **YOUR_CLOTHING_ITEM** with the desired name for your clothing, such as 'Shirt', 'GenericTop1', Skirt', 'Pants', etc. Each clothing item needs to have a unique name. We will return to these clothing classes later.

The '**AllowedMainClothingTypes**' is a list of all allowed clothes your race can wear. This list can contain any clothing function from any race file. So, if you used a specific race as a base for your body, you can include their generic clothing in this list as extra options.

Note: When using clothes that are not randomly generated ensure the number of them are indicated '**AvoidedMainClothingTypes**' variable and are at the bottom of the list.

Use '**AllowedWaistTypes**' the same way as above for main clothing, but for the waist slot, Do the same for '**ExtraMainClothingTypes**' with additional clothing items, which can be things like necklaces, scarves, or pretty much any kind of accessories you made sprites for.

Example '**AllowedMainClothingTypes**' and '**AllowedWaistTypes**'

```
AllowedMainClothingTypes = new List<MainClothing>()
{
    new GenericTop1(),
    new GenericTop2(),
    new GenericTop3(),
    new GenericTop4(),
    new GenericTop5(),
    new GenericTop6(),
    new GenericTop7(),
    new MaleLowClass(),
    new MaleHighClass(),
    new MaleLightArmor(),
    new MaleHeavyArmor(),
    new FemaleTunic(),
    new FemaleDress(),
    new FemaleLightArmor(),
    new FemaleHeavyArmor(),
    new FemalePriestess(),
    new Natural(),
    Rags,
};

AvoidedMainClothingTypes = 1;
```

```

AllowedWaistTypes = new List<MainClothing>()
{
    new GenericBot1(),
    new GenericBot2(),
    new GenericBot3(),
    new GenericBot4(),
    new GenericBot5(),
    new Loincloth(),
};

```

Each item is a clothing item defined at the bottom of the file. Note how Rags is the last item with one '**AvoidedMainClothingTypes**'. This means the Rags clothing will not be generated randomly.

Offsets

The next function '**SetBaseOffsets()**' can be used to change the position of any of the above, this way it's easy to manipulate the position of sprites with logic, instead of modifying the position of sprites. This function can do things such as raise the position of eyes by 1 if the unit is female. Look at the files for other races if you need an example.

IMPORTANT: When using offsets, ensure all values are **multiplied by '625f'**

Example:

```

1. AddOffset(Belly, 0, -1 * .625f); // Moves breast sprites down one pixel
2. AddOffset(Weapon, 5 * .625f, 0); // Moves weapons right five pixels

```

Randomization

The '**RandomCustom()**' function is where all of the logic for the building of the unit is held. If you are making anything that is not a special mercenary include the following at the very top of your '**RandomCustom()**' function. This base function is an abstraction that provides the minimum for randomization.

```

base.RandomCustom(unit);

```

If you have leader specific clothing, ensure you include a check for the leader unit type, which looks something like this:

```

1. if (unit.Type == UnitType.Leader)
2. {
3.     ...
4. }
5.

```

If your race has rags sprites for when they are recruited by an enemy empire. The code for that looks like this, assuming the rags clothing item is named 'Rags' in your main clothing list:

```

1. if (Config.RagsForSlaves && State.World?.MainEmpires != null &&
    (State.World.GetEmpireOfRace(unit.Race)?.IsEnemy(State.World.GetEmpireOfSide(unit.Side)) ?? false)
    && unit.ImmuneToDefections == false)
2. {
3.     unit.ClothingType = 1 + AllowedMainClothingTypes.IndexOf(Rags);
4.     if (unit.ClothingType == 0) //Covers rags not in the list
5.         unit.ClothingType = AllowedMainClothingTypes.Count;
6. }

```

'RandomCustom()' only uses variables defined above it, you may need to add or remove lines to include the proper sprites if you copied it from another race file. We'll skip over for now and come back later when everything else is set up.

Simple Part Functions

Now, we are onto the Sprite returning functions I mentioned at the end of the part definition section. Using the sprite variables defined at the top of the file, we can treat the sprite sheet as a list, where 0 is the top left most sprite. **It can help to have your sprite sheet or Unity open in another window as you will need to have the index for each sprite known.**

With this knowledge, you can point the function to the correct sprite on the sheet by counting from left to right starting at 0. Reference some already made races code and sprite sheets with this in mind to better understand what's going on. I will try my best to explain it here.

This is the code for the Mice's tail sprite:

```
protected override Sprite BodyAccentSprite2(Actor_Unit actor) => Sprites[44 + actor.Unit.TailType];
```

As you can see it's a simple one-line function. The name of the function, 'BodyAccentSprite2' Refers to the same function in the definition. 'Sprites' refers to the ambiguously named variable I had defined earlier. **44** references the index of the first tail sprite the mice have on their sprite sheet.

You may have noted the 44 is being added to by some kind of actor variable. Actor is an argument received by all sprite functions and it refers to the 'actor' for the unit shown in the game. Earlier, I defined **TailTypes = 4;** when making definitions. This caused the unit to choose a tail type between 0 and 3 when it was first generated by 'RandomCustom()'. Therefore, the range of values to be looked at can be from **44 – 47**. If you review the sprite sheet '**Mice1**' in Unity, sprites 44, 45, 46, and 47 are all tails for the mice.

These part functions work best for parts shared by the entire race that are either a single sprite, or gender neutral items like ears or horns that don't need any logic.

Complex Part Functions

Some expressions can seem rather complex, especially if you aren't familiar with C# coding. These complex functions are the backbone of race sprites, and make up most of the work when coding races.

Unfortunately, it is not easy to simply explain these functions, as they all vary and have different objectives, so it is best to review the way other races make their implementations. Instead, I will go over some common things that show up in these functions.

Some confusion can come from inline conditionals.

This code is of the form (A == B ? x : y)

This inline truth statement is equivalent to if(A == B) X; else Y;

It is very helpful for checking if a unit uses a female body or has a specific value.

For example, this code checks if a unit has an accent greater than two and if they are female.

```
return Sprites[50+(actor.Unit.BodyAccentType3 > 2 == 0 ? 4 : 0)+(actor.Unit.HasBreasts ? 12 : 0)];
```

These kinds of statements are used many times, as they can be used inside of list indices and back to back, to create simple looking functions, or turn a complex part function into a more simple one.

When working with arm and weapon sprites you may run into a **switch**, the function 'GetWeaponSprite()' returns an integer 0-7, depending on the current state of weapons. These numbers are as follows:

0 - Light melee idle

1 - Light melee attack

2 - Heavy melee idle

3 - Heavy melee attack

4 - Light ranged idle

5 - Light ranged attack

6 - Heavy ranged idle

7 - Heavy ranged attack

Be sure to line these up with whatever sprites you may have used to create your arm, clothing, or weapon's idle and attack sprites.

The next hump is likely going to be the belly sprites. If your race has specific belly sprites when eating other races, ensure they are included in this function.

Ensure that if you have the proper offset and it is multiplied by .625f

To test offsets for your sprites, you can guess and check by changing values and looking in unity to ensure they look how you want them to.

Coding Outfits

With all of the body sprites coded in, it's time to add outfits. Tops or full sets of clothing should be included in the '**AllowedMainClothingTypes**' list. Clothing items included in '**AllowedWaistTypes**' should be clothes that cover the waist. Each piece of clothing has it's own class, which is inherited from the MainClothing class. While **VoreWar\Assets\Scripts\Utility\MainClothing.cs** has every option for MainClothing, you may not want to use or change them. I'll go over them below.

All variables are the default values, if you want to change the value, add the variable to your class with the correct value. If not, don't include it in the clothing class you make:

'leaderOnly = false;' means the clothing piece is only for leaders. If true, this clothing class should not be defined within AllowedMainClothingTypes, as it needs to be used later

'femaleOnly = false; or maleOnly = false;' decides if the clothing is gender locked.

'breastSprite = null;' will replace the character's breast sprite, used for cleavage windows or other such things

'weaponOverride = null;' will replace the character's normal weapon sprite when they wear the outfit

'ReqWinterHoliday = false;' if true, the clothing will only be available during the Christmas holiday season

'OccupiesAllSlots = false;' if true, will prevent waist clothing from rendering, used for full sets of clothing, such as a shirt and pants combo, or leotard

'DiscardUsesPalettes = false;' if true, the discarded sprite uses palettes instead of the solid color

'coversBreasts = true;' if true, lowers breast layer to 8 so that it will be under clothing

'blocksBreasts' will turn off the breast sprite completely, this has no default value and will need to be set when the class is created.

'blocksDick = true;' will turn off the dick and balls sprites entirely, useful for pants or if using bulge sprites

'inFrontOfDick = false;' will render the clothing in front of dick sprites, useful for loincloths

'colorsBelly = false;' decides if the color below will apply to the belly.

'bellyColor = Color.white;' dictates what color will be applied to the belly if the above is true.

'bellyPalette = null;' dictates what palette the belly will use if the above is true

'HidesFluff = false;' if true, will remove the arm/leg fluff graphic for furry races, useful for sleeves or pants

'FixedColor = false;' if true, clothing is always the default color, also affects the discard

'DiscardSprite = null;' should be set to whatever your discard sprite is for the clothing

'Type' is a unique type number, only used in relation to discard sprites, I recommend a random sequence of numbers you increment for each piece of your clothing. It must be unique to other clothing items.

'DiscardUsesColor2 = false;' dictates whether discards will use clothing 2 color instead of clothing color 1

'clothing1' - 'clothing9' are all pieces of clothing. These are used similarly to the part definitions.

For example, a bikini top will only use 'clothing1' while a full set in one outfit could use 'clothing1' for the shirt, 'clothing2' for the pants, 'clothing3' for the shoes, etc. Each variable will need to be set equal to 'SpriteExtraInfo(int, Sprite, Color);', this dictates, they layer the sprite will be rendered on, and it's color.

Color is usually set to **null** (color changed by palette) or **WhiteColored** (color does not change). For each of these you define, you should have a definition for, or else the sprite will not show. For your own reference it can be helpful to label these yourself with comments.

Example:

```

class FemaleDress : MainClothing
{
    public FemaleDress()
    {
        DiscardSprite = State.GameManager.SpriteDictionary.Aviants4[79];
        femaleOnly = true;
        coversBreasts = false;
        blocksDick = true;
        OccupiesAllSlots = true;
        clothing1 = new SpriteExtraInfo(15, null, WhiteColored); //belly part
        clothing2 = new SpriteExtraInfo(19, null, WhiteColored); //breast part
        clothing3 = new SpriteExtraInfo(16, null, WhiteColored); //right arm
        clothing4 = new SpriteExtraInfo(6, null, WhiteColored); //shoes
        Type = 1579;
        DiscardUsesPalettes = false;
        SwapedClothing = 9;

    }
    . . .
}

```

Sprite definition are done in the Configure() class

Here your clothing should include conditionals for what you made sprites for example, a check for if an actor has a full belly would use an 'if (actor.HasBelly)' conditional.

Here are some key things to consider:

1. You can perform math in the sprite sheet index:

Here are some values that are commonly used in conditionals and sprite definitions

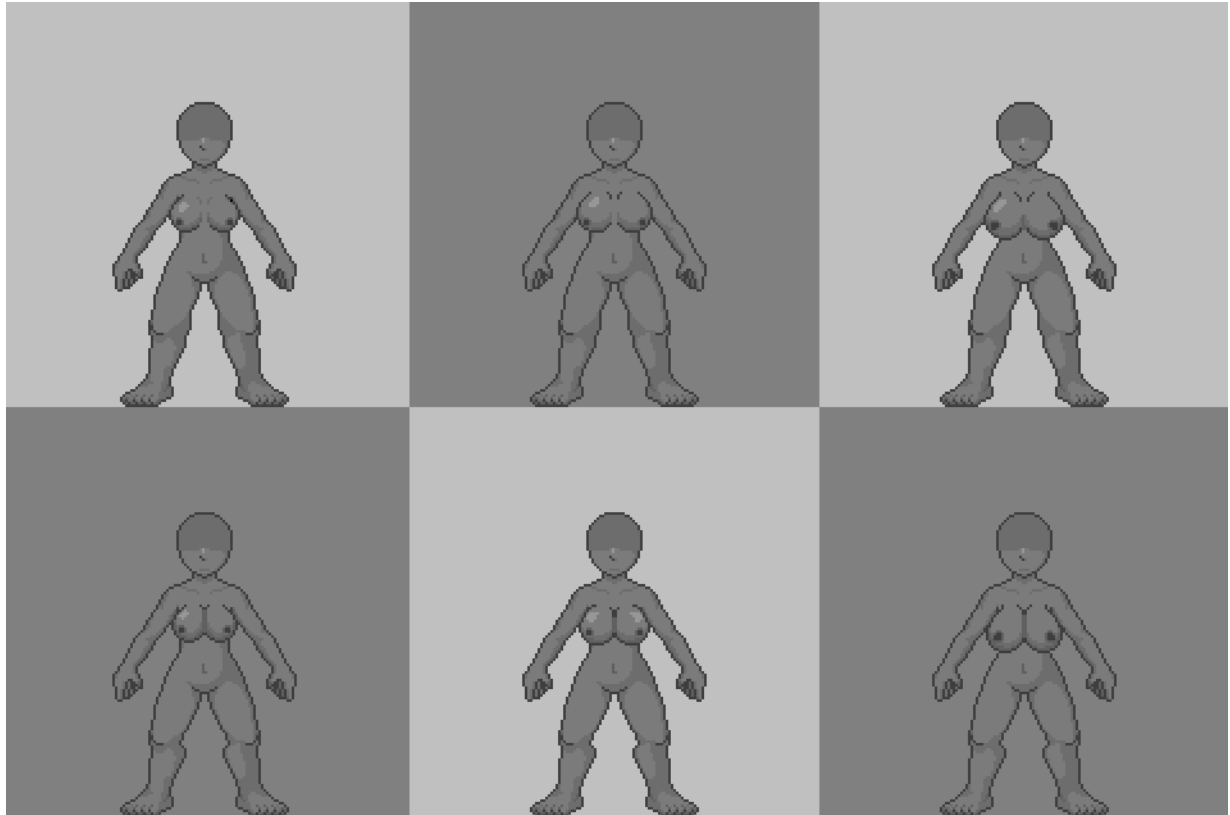
actor.Unit.BodySize - Will use the current unit's bodysize

Oversize is a variable set to true when your breast size is over the cap you defined earlier in the file.

This is usually useful when dealing with breast vore sprites. You can use

'Races.YOUR_RACE.oversize' in a conditional to apply your sprites. Your race must have the oversize variable defined to use this.

Something to note is the need to use different breast sprites with your clothing, such as getting an effect of tight clothing squeezing breasts together or have a pushup effect. To achieve this, ensure **'blocksBreasts'** is set to true.



An example of squeezing on sizes 4,5,& 6.

Make a conditional for every size you made. Target the sizes your squeezed together sprites are, for most, this is sizes 3, 4, and 5. Ensure your replacement breasts sprites are both included and have a lower clothing layer. Keep in mind you will need two clothing slots to achieve this, leaving you with 6 others for your actual clothing, so you may need to condense your sprites if you run out. Review already coded clothing for examples of this.

If your clothing has sprites related to the arms, they will need to be defined here as well. Don't worry, you can use the same function which changes arm sprites from earlier in the file.

You'll need to change up the return values to their dynamic variants like so:

'return Sprites3[100 + actor.Unit.BodySize]', where 'Sprites3' is 'YOURSHEETNAME',

Would become

'clothingX.GetSprite = (s) => State.GameManager.SpriteDictionary.YOURSHEETNAME[100 + actor.Unit.BodySize];'

Once you're done, ensure your Configure() function ends with **'base.Configure(sprite, actor);'** If you copied a clothing class, this shouldn't be a problem.

Custom Projectile

If your race has their own projectile sprites, say a rock instead of an arrow, here's how to change it

navigate to VoreWar\Assets\Scripts\Modes\Tactical\TacticalGraphicalEffects.cs

find the function 'ArrowType(Actor_Unit actor, out Material material)'

Follow the current implementation and return the index of your projectile in the same way you added other spites.

Adding palettes

Let's add functionality to your palette. "Wait! What the hell is a palette?" I hear you asking.

Well, get yourself a glass of water, a nice snack, and a comfy chair, because palettes are one of the more complicated features. Go ahead and grab that snack if you want it, I'll wait. Just follow these steps and you'll (hopefully) avoid groping around in complete darkness.

Navigate to VoreWar\Assets\Scripts\Utility\PaletteDictionary.cs

Give your palette a name. Usually we use the same name as the actual file. **Add your palette to the end of the dictionary in the same fashion as the others.** This is the format Unity uses to read your palette.

Navigate to VoreWar\Assets\Scripts\Utility\ColorPaletteMap.cs

Add your palette to the end of the 'SwapType' enum.

Define your palette like the others:

```
List<ColorSwapPalette> NAME_OF_PALETTE_Swaps = WireUp(SwapType.NAME_OFPALETTE);
```

Your palette colors will be defined like so:

'[A] = map.GetPixel(B, pixelY);'

A is an red value number that can range between 0 and 255. If your color is not red, such as a shade of grey, it will still have an R value you can use. You can find what this value is by checking the color in the same program you made your sprites in.

B is the index of the pixel on your palette image, with the top left most pixel being 0.

Repeat for each palette file you made.

Ensure you return to your main race file check that your clothes, hair, Accessories, and skin use the correct palette.

Lets look at the pallet function 'ColorPaletteMap.GetPalette(ColorPaletteMap.SwapType.XXX, actor.Unit.YYY);'

The first argument takes a the SwapType of your palette, the one you just defined, make sure you use the palette you made, or if applicable, use a preexisting one, such as 'Skin' for skin color.

The second argument is what the unit will select their palette off of in the game's editor menu, these may sound familiar, being: 'AccessoryColor', 'HairColor', 'SkinColor', 'ClothingColor'

Once you've set these values, if you used 'ColorPaletteMap.GetPaletteCount()' to count out your palette, you're all done.

See, that wasn't so bad, now your race has varied colors. If you ever want to add a new color, add a new row onto your pallet sheet, the above functions will handle everything automatically.

Adding your sprites to Unity

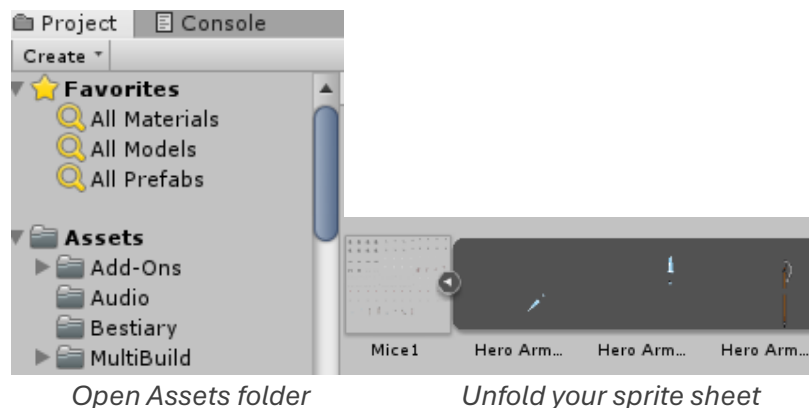
There's going to be a few jumps between unity and your IDE, so make sure to keep both open while you go through this section.

If you're making an empire race do the following:

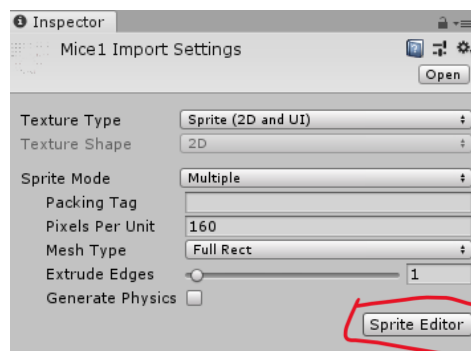
- Navigate to Config.cs
- Find NumberOfRaces, then increment by one

Open Unity.

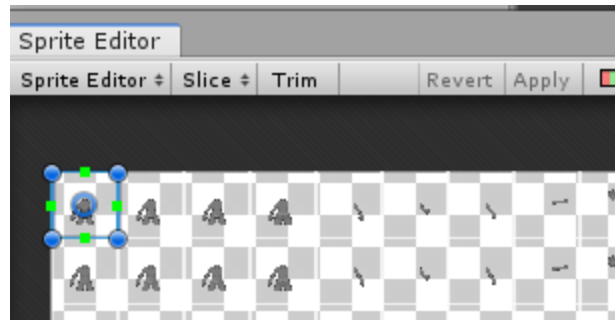
To start, we're going to slice up our sprites. Look to the **Project** tab at the bottom, **ensure your Assets folder is open**. Look for the '**Sprites**' folder and find where you put your race. This folder setup is exactly how it looks in your file browser. Once you've found your files, If you click the arrow, you'll see pre-cut sprites named 'Hero Armor_X' these are auto generated and can be ignored.



Select your sprite and look at the inspector, which defaults to the right side of the screen. Select the 'Sprite Editor' to open the sprite editor window. Within the window, look to the top. You should see 'Slice' or 'Sl' if your window is not wide enough.



Inspector on the right side of the Unity window.



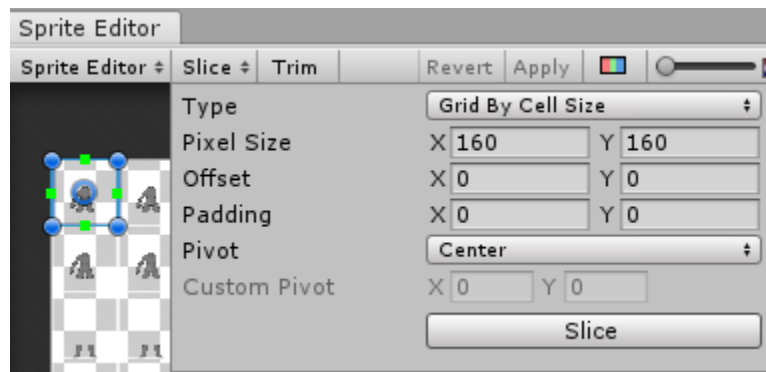
Sprite editor window

Select Slice

Change 'Type' to 'Grid By Cell Size'

Change your 'Pixel size' to the size of your sprite

Select 'Slice'



Setup for Mice sprites

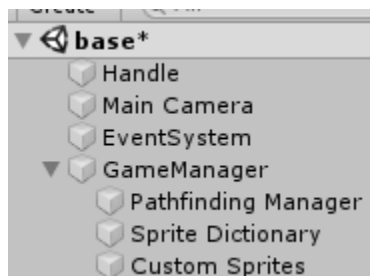
If everything looks good, select 'Apply' at the top

Repeat for each of your sprite sheets

Now, let's add our sprites to the dictionary.

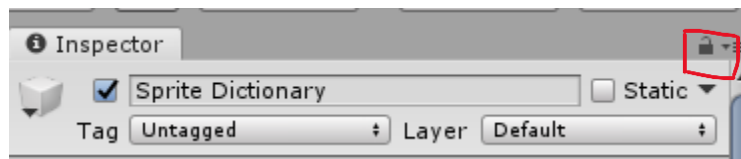
Within the hierarchy, Open the GameManager box.

Click on 'Sprite Dictionary' so it is selected.

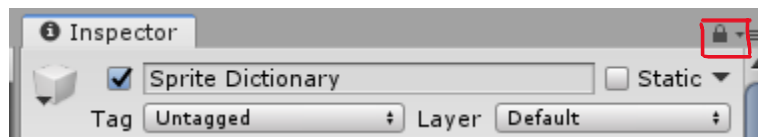


Search for the Sprite Dictionary

Lock your inspector by clicking the lock in the top right corner



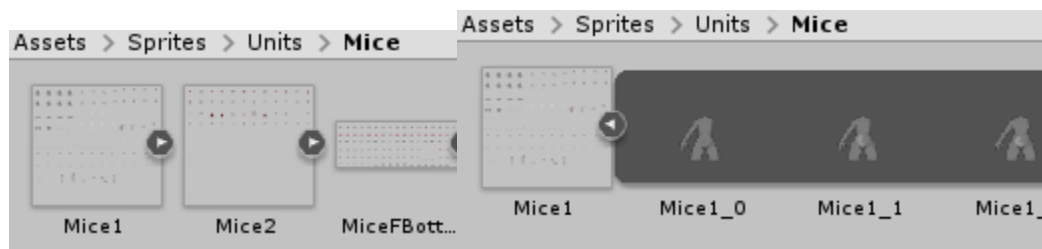
An Unlocked Inspector



Locked Inspector

Scroll to your dictionary entry, it will have the same name and position as the one defined in the file.

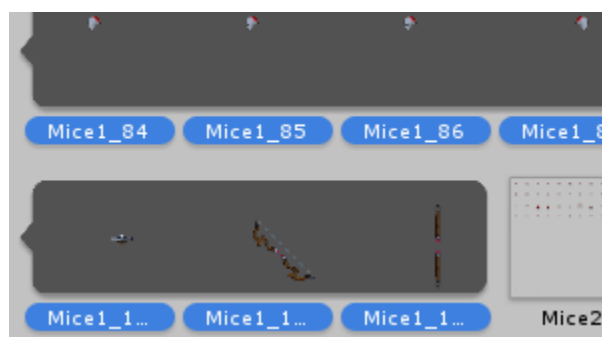
Within the Project window, access the cut sprites by clicking the arrow.



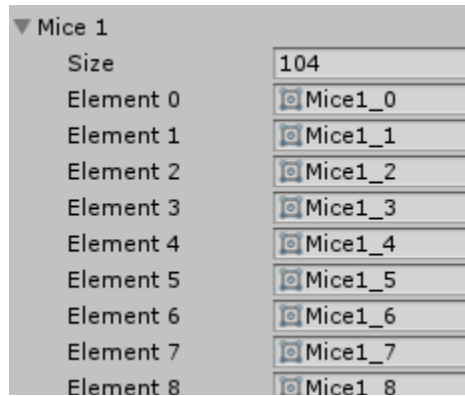
Click the first sprite. It will be named the same as the vile with an '_0' Suffix.

Scroll to the end of your cut sprites and shift + click the last sprite so that every sprite is selected

Drag and drop the selected sprites under the correct dictionary element



All Sprites highlighted



Sprites placed into correct Dictionary entry

Repeat for all sprite sheets

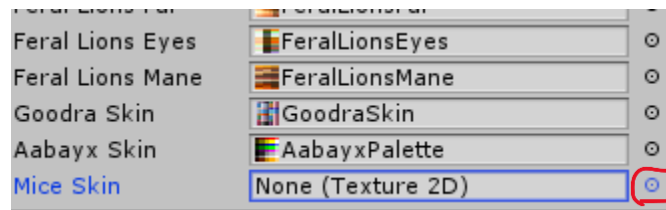
If you mess up, change the 'size' at the top of the list to '0', then click away or press enter on your keyboard. This will clear all elements and reset the entry.

Remember to unlock your inspector.

If you added your own pallet, we need to add that as well

Select 'GameManager' in the hierarchy window.

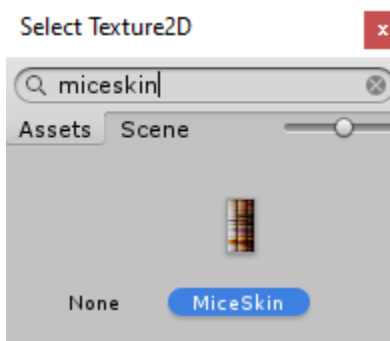
Scroll to your entry under the Palette Dictionary script, it will have the same name and position as the one defined in the file



Entry in GameManager

Update the entry by selecting the small circle

In the new window, search for the file name of your pallet



Double click the correct file

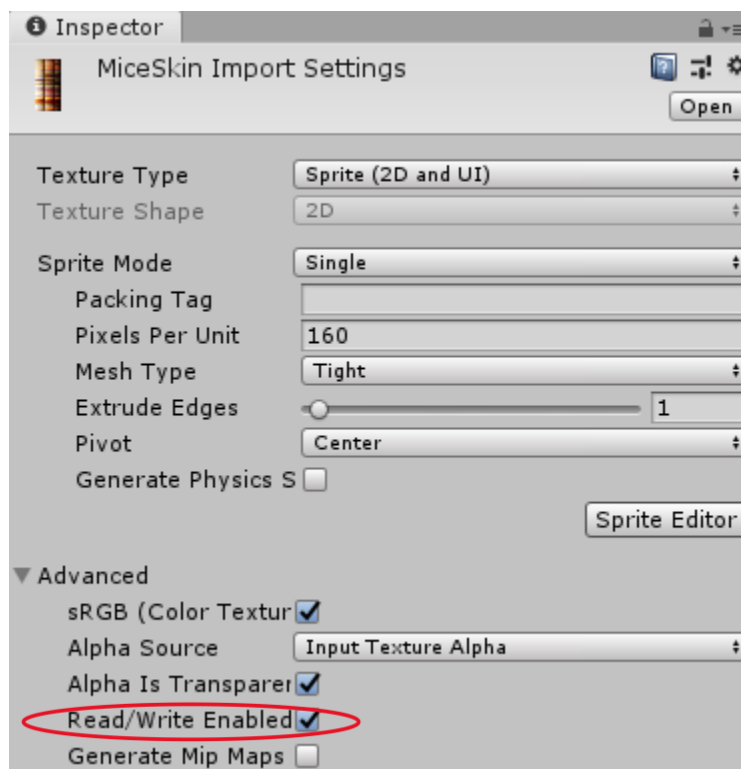
Look at the assets folder in the project window.

Open the resources folder.

Click the ColorPalettes folder

Locate your Palette and click it.

Look under the 'Advanced' Section in the 'Inspector' and ensure Read/Write Enabled has a check.



The following is only required if you are creating an empire race. If creating a normal or special mercenary race, you are done.

Click the '**sprites**' folder in the project window

Click **banners**, then slice them like you did your race sprites. The size of each banner is 64x64, so use that number.

Once sliced, click '**strategic**' in the hierarchy tab

In the inspector, click '**Banners**'

Lock the inspector

Click on Banners_0 to be taken to the banner sprite file.

Set the 'Banners' size to 0, press enter on your keyboard.

Click the first banner sprite

Scroll to the end of your cut banner sprites and shift + click the last sprite so that every sprite is selected

Drag and drop onto 'Banners'

Unlock the inspector

It is now time to add the village to the Map Editor.

In the hierarchy window, click 'Map Editor'

Open 'Map editor UI'

Open 'Panel'

Open 'Empire Panel'

Scroll down to the most recently added main race, above 'Vacant'

Duplicate 'Vacant' by right clicking it or selecting it and pressing 'ctrl + D'

Drag the duplicated 'Vacant (1)' above 'Vacant'

Rename 'Vacant (1)' to your race

In the inspector change the village graphic to your race's unwall'd village.

Scroll down and change the race under 'Map Editor Village (Script)' to your race.

Coding a trait

Traits are rather variable in their difficulty to code into the game. Depending on how complex your trait is supposed to be, it'll take lots of work, or almost none. Regardless of what your trait is going to do, the start is always the same.

Firstly, Navigate to **VoreWar\Assets\Scripts\Enums\Tags.cs**

Here, you'll see a massive list of traits, as it's the Enum list for every trait in the game. Depending on the trait's kind, you'll add it in a different location. Traits are organized in this way so their type can be easily referenced, so ensure you put your trait in the correct area.

Use the comment text to find the correct place for your trait and add it to the list, set it equal to the next number in line. Additionally, add the description for your trait as the others do above it. For example:

```
/// <summary>Unit's attacks also have Mind Stat Scaling.</summary>
SpellBlade = 176,
/// <summary>Unit gains 1 focus when landing a spell, gains 4...summary>
ArcaneMagistrate = 177,
/// <summary>Damage over time effect received from being devoured by an Aabayx</summary>
ViralDigestion = 178,
/// <summary>This unit has a very strange body type...</summary>
```



```
AwkwardShape = 179,
```

Would become

```
/// <summary>Unit's attacks also have Mind Stat Scaling. </summary>
SpellBlade = 176,
/// <summary>Unit gains 1 focus when landing a spell, gains 4...</summary>
ArcaneMagistrate = 177,
/// <summary>Damage over time effect received from being devoured by an Aabayx</summary>
ViralDigestion = 178,
/// <summary>This unit has a very strange body type...</summary>
AwkwardShape = 179,
/// <summary>THE DESCRIPTION OF THE TRAIT</summary>
TRAIT_NAME = 180,
```

Note how the new trait is added with its own summary. Add any other traits you have to this list.

Next navigate to **VoreWar\Assets\Scripts\Races\Traits.cs**

Here is where you add functionality to the simpler traits. Anything within the **PermanentBoosts** class are things that you can change with no extra work, as their effects are already coded and meant to be changed here.

Scroll down to the **TraitList** class.

As you can see, there are a few different types of traits. The biggest one is the **Booster()** type, which affects the variables in the **PermanentBoosts** class from above.

Making options

1. Make setting in code

Add the value to Config.cs

Add value to WorldConfig.cs if necessary (any setting that can change per save), below the comment

Most Bool types should be added to the Dict in the ResetDictionary() function at the bottom of the file

Anything else can be done the same as other options of the same type

Set it's default value accordingly

Add the object to ContentSettings.cs

Bools are of Toggle type

Everything else is either of Slider, TMP_Dropdown, or InputField type. Decide which one for yourself

In the Refresh(), function, the number values are the defaults, set yours accordingly

Follow along with what other objects do in ContentSettings.cs

2. Add it to Unity

Open Unity

Navigate to Game Menu -> Content Settings -> Whatever Button page you plan to put the setting onto

Copy paste an existing option matching the same type you want to add

Change the values (Name, Min, max, etc.)

Go back to the Game Menu -> Content Settings clicking the Content Settings block

Look in the inspector for your setting you made in ContentSettings.cs it should say something like 'None (Slider)

Click the circle next to the box and search for the Name of your slider object you made in unity

Click Content Settings in the Hierachy

In the inspector scroll down to your setting

Link the setting to your unity object

Note: If the slider doesn't show up, make sure to check the scale of the object.

Note: You can adjust the position of your option by dragging it higher in the heirachy

3. Add your tooltip

Navigate to DefaultTooltips.cs

Scroll to the bottom

Add another case

In the unity inspector, scroll to the bottom

match the 'display tooltip' with your case number

Army MP	Army MP (Slider)	⊙
Army Creation MP M	None (Slider)	⊙
Army Creation MP C	None (Slider)	⊙
Custom Event Frequ	Custom Event Slider (Slider)	⊙

Army MP	Army MP (Slider)	⊙
Army Creation MP M	Army Creation MP Modifier (S	⊙
Army Creation MP C	Army Creation MP Curve (Slic	⊙
Custom Event Frequ	Custom Event Slider (Slider)	⊙